

1.

```
1  Algorithm InverseRotation(A, n)
2    for i = 0 to n - 2 do
3      if A[i] > A[i + 1] then
4        return i + 1
5    end for
6    return 0
7  end Algorithm
8
```

2.

Linear Probing

Formula: $h(k) = k \bmod 11$, $h_i(k) = (h(k) + i) \bmod 11$

Key	$H(k) = k \bmod 11$	Computation steps	Final index
10	10	$(10 + 0) \% 11 = 10$	10
22	0	$(22 + 0) \% 11 = 0$	0
31	9	$(31 + 0) \% 11 = 9$	9
4	4	$(4 + 0) \% 11 = 4$	4
15	4	$(15 + 0) \% 11 = 4 \rightarrow$ $(15 + 1) \% 11 = 5$	5
28	6	$(28 + 0) \% 11 = 6$	6
17	6	$(17 + 0) \% 11 = 6 \rightarrow$ $(17 + 1) \% 11 = 7$	7
88	0	$(88 + 0) \% 11 = 0 \rightarrow$ $(88 + 1) \% 11 = 1$	1
59	4	$(59 + 0) \% 11 = 4 \rightarrow$ $(59 + 1) \% 11 = 5 \rightarrow$ $(59 + 2) \% 11 = 6 \rightarrow$ $(59 + 3) \% 11 = 7 \rightarrow$ $(59 + 4) \% 11 = 8$	8

Final Hash Table

index	Key
0	22
1	88
2	-
3	-
4	4
5	15
6	28
7	17
8	59
9	31
10	10

In linear probing, collisions are resolved by checking the next slot in sequence, which can cause clustering.

Quadratic probing

Formula: $h(k) = k \bmod 11$, $h_i(k) = (h(k) + c_1 i + c_2 i^2) \bmod 11$, where $c_1 = 1$, $c_2 = 3$

Key	H(k)	Computation steps	Final index
10	10	$(10 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 10$	10
22	0	$(0 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 0$	0
31	9	$(9 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 9$	9
4	4	$(4 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 4$	4
15	4	$(4 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 4 \rightarrow (4 + 1 \cdot 1 + 3 \cdot 1^2) \% 11 = (4 + 4) \% 11 = 8$	8
28	6	$(6 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 6$	6
17	6	$(6 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 6 \rightarrow (6 + 1 \cdot 1 + 3 \cdot 1^2) \% 11 = 10 \rightarrow (6 + 1 \cdot 2 + 3 \cdot 2^2) \% 11 = (6 + 2 + 12) \% 11 = 9 \rightarrow (6 + 1 \cdot 3 + 3 \cdot 3^2) \% 11 = (6 + 3 + 27) \% 11 = 3$	3
88	0	$(0 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 0 \rightarrow (0 + 1 \cdot 1 + 3 \cdot 1^2) \% 11 = 4 \rightarrow (0 + 1 \cdot 2 + 3 \cdot 2^2) \% 11 = 3 \rightarrow (0 + 1 \cdot 3 + 3 \cdot 3^2) \% 11 = 8 \rightarrow (0 + 1 \cdot 4 + 3 \cdot 4^2) \% 11 = (0 + 4 + 48) \% 11 = 2$	2
59	4	$(4 + 1 \cdot 0 + 3 \cdot 0^2) \% 11 = 4 \rightarrow (4 + 1 \cdot 1 + 3 \cdot 1^2) \% 11 = 8 \rightarrow (4 + 1 \cdot 2 + 3 \cdot 2^2) \% 11 = (4 + 2 + 12) \% 11 = 7$	7

Final Hash Table

index	Key
0	22
1	-
2	88
3	17
4	4
5	-
6	28
7	59
8	15
9	31
10	10

In quadratic probing, the offset increases quadratically ($i + 3i^2$), distributing keys more evenly and reducing clustering.

3.

case	Best algorithm strategy	Why
A	Brute force	You'd have to try every possible combination of characters to see if the password can be cracked, so brute force makes sense
B	Divide and conquer	The images are really big, so it's easier to split them into smaller parts and work on each part separately, that's divide and conquer.
C	Divide and conquer	Since there's a ton of data, it's better to break it up into smaller chunks to process faster, instead of doing it all at once
D	Data structures	When you click "back," you're just going to the last page you visited, stacks are good for that since they store things in order
E	Queue	The calls have to be handled in the same order they came in, which is exactly how a queue works (first in, first out).
F	Priority queue/ heap	Patients with more serious problems need help first, so using a priority queue makes sense since it sorts by urgency.
G	Hash table	You want quick suggestions as you type, so using a hash table or trie lets the computer find results really fast
H	Priority queue/ heap	People with higher status (like first class) should get their seats first, so a priority queue works best
I	Hash table + cache	The app needs to quickly show songs you listen to a lot, so it helps to store them in a hash table or cache for fast access
j	Divide and conquer / hashing	Comparing every patient and donor one by one would take forever, so breaking the data up by criteria makes it much faster